

CÓDIGOS PARA PROVA:

-- implementacao dos multiplexadores 4x1 para revisao
-- disponivel no registro da disciplina ELD I do professor marcos moecke

```
library ieee;  
use IEEE.STD_LOGIC_1164.all;
```

```
entity multiplexadores is  
    port  
    (  
        X: in  bit_vector (3 downto 0);  
        Sel : in bit_vector (1 downto 0);  
        Y : out bit  
    );  
end entity;
```

```
architecture v_logica_pura of multiplexadores is  
begin  
    Y <= (X(0) and (not Sel(1)) and (not Sel(0))) or  
        (X(1) and (not Sel(1)) and Sel(0)) or  
        (x(2) and Sel(1) and not (Sel(0))) or  
        (x(3) and Sel(1) and Sel(0));  
end architecture;
```

```
-- IMPLEMENTACAO WHEN ELSE  
architecture v_WHEN_ELSE of multiplexadores is  
begin  
    Y <= X(0) when Sel = "00" else  
        X(1) when Sel = "01" else  
        X(2) when Sel = "10" else  
        X(3);  
end architecture;
```

```
-- IMPLEMENTACAO WITH SELECT  
architecture v_WITH_SELECT of multiplexadores is  
begin  
    with Sel select  
    Y <= X(0) when "00",  
        X(1) when "01",  
        X(2) when "10",  
        X(3) when others; -- note o uso de others, para todos os demais valores.  
        -- Não pode ser substituído por "11" mesmo que o signal seja bit_vector.  
end architecture;
```

```
configuration cfg_ifsc_v1 of multiplexadores is  
    for v_logica_pura end for;
```

```

        for v_WHEN_ELSE end for;
        for v_WITH_SELECT end for;
end configuration;

```

O que é: O código descreve um **Multiplexador 4x1**, que funciona como uma "chave seletora" digital (usa 2 pinos para escolher 1 entre 4 entradas e enviá-la para a saída).

A Entidade (entity): Define os pinos físicos do chip. X são as 4 opções de entrada, Sel são os 2 pinos seletores, e Y é a saída.

As Arquiteturas (architecture): O professor mostrou 3 jeitos diferentes de programar a mesma lógica:

Lógica Pura: Forma de baixo nível, usando a matemática direta da tabela-verdade (portas AND, OR, NOT).

When / Else: Forma comportamental. Funciona como um "Se / Senão" convencional, sendo bem mais fácil de ler.

With / Select: Funciona como um "Switch / Case". *Atenção:* O uso do others na última linha é obrigatório no VHDL para garantir que não sobrem opções vazias, o que causaria erros de memória no circuito.

A Configuração (configuration): É apenas um menu final para você dizer ao compilador qual das três arquiteturas acima você quer gravar no chip definitivo.

MUX .DO V1

```

vlib rtl_work
vmap work rtl_work

vsim work.multiplexadores(v_logica_pura)

add wave -position insertpoint \
sim:/multiplexadores/X \
sim:/multiplexadores/Sel \
sim:/multiplexadores/Y

force -freeze sim:/multiplexadores/Sel 00 0
force -freeze sim:/multiplexadores/X 0001 0
run
force -freeze sim:/multiplexadores/X 0010 0
force -freeze sim:/multiplexadores/Sel 01 0
run
force -freeze sim:/multiplexadores/X 0100 0
force -freeze sim:/multiplexadores/Sel 10 0
run
force -freeze sim:/multiplexadores/X 1000 0
force -freeze sim:/multiplexadores/Sel 11 0
run

```

MUX .DO V2

```

vlib rtl_work
vmap work rtl_work

vsim work.multiplexadores(v_WHEN_ELSE)

```

```

add wave -position insertpoint \
sim:/multiplexadores/X \
sim:/multiplexadores/Sel \
sim:/multiplexadores/Y

force -freeze sim:/multiplexadores/Sel 00 0
force -freeze sim:/multiplexadores/X 0001 0
run
force -freeze sim:/multiplexadores/X 0010 0
force -freeze sim:/multiplexadores/Sel 01 0
run
force -freeze sim:/multiplexadores/X 0100 0
force -freeze sim:/multiplexadores/Sel 10 0
run
force -freeze sim:/multiplexadores/X 1000 0
force -freeze sim:/multiplexadores/Sel 11 0
run

```

MUX .DO V3

```

vlib rtl_work
vmap work rtl_work

vsim work.multiplexadores(v_WITH_SELECT)

add wave -position insertpoint \
sim:/multiplexadores/X \
sim:/multiplexadores/Sel \
sim:/multiplexadores/Y

force -freeze sim:/multiplexadores/Sel 00 0
force -freeze sim:/multiplexadores/X 0001 0
run
force -freeze sim:/multiplexadores/X 0010 0
force -freeze sim:/multiplexadores/Sel 01 0
run
force -freeze sim:/multiplexadores/X 0100 0
force -freeze sim:/multiplexadores/Sel 10 0
run
force -freeze sim:/multiplexadores/X 1000 0
force -freeze sim:/multiplexadores/Sel 11 0
run

```

CÓDIGO 2

```

library ieee;
use IEEE.STD_LOGIC_1164.all;
use ieee.numeric_std.all;

```

```

entity atribuicao_sequential is
    port

```

```

(
    a : in std_logic_vector(7 downto 0);
    b : in std_logic_vector(7 downto 0);

    gt : out std_logic;
    eq : out std_logic;
    lt : out std_logic
);
end atribuicao_sequencial;

-- implementacao atribuicao incorreta
architecture atribuicao_incorreta of atribuicao_sequencial is
    signal a_signed, b_signed : signed(7 downto 0);
begin
    a_signed <= signed(a);
    b_signed <= signed(b);

    process (a_signed,b_signed)
    begin
        if (a_signed > b_signed) then
            gt <= '1';
        elsif (a_signed = b_signed) then
            eq <= '1';
        else
            lt <= '1';
        end if;
    end process;
end atribuicao_incorreta;

-- correcao 1 para o codigo acima
architecture atribuicao_correta_v1 of atribuicao_sequencial is
    signal a_signed, b_signed : signed(7 downto 0);
begin
    a_signed <= signed(a);
    b_signed <= signed(b);

    process (a_signed,b_signed)
    begin
        if (a_signed > b_signed) then
            gt <= '1';
            eq <= '0';
            lt <= '0';
        elsif (a_signed = b_signed) then
            gt <= '0';
            eq <= '1';
            lt <= '0';
        else
            gt <= '0';

```

```

        eq <= '0';
        lt <= '1';
    end if;
end process;
end atribuicao_correta_v1;

-- correcao 2 para o codigo acima
architecture atribuicao_correta_v2 of atribuicao_sequencial is
    signal a_signed, b_signed : signed(7 downto 0);
begin
    a_signed <= signed(a);
    b_signed <= signed(b);

    process (a_signed,b_signed)
    begin
        gt <= '0';
        eq <= '0';
        lt <= '0';

        if (a_signed > b_signed) then
            gt <= '1';
        elsif (a_signed = b_signed) then
            eq <= '1';
        else
            lt <= '1';
        end if;
    end process;
end atribuicao_correta_v2;

configuration cfg_atribuicoes of atribuicao_sequencial is
    --for atribuicao_incorreta end for;
    --for atribuicao_correta_v1 end for;
    for atribuicao_correta_v2 end for;
end cfg_atribuicoes;

```

Esse código descreve um **Comparador Digital**. Ele recebe dois números (a e b) e avisa se a é maior, igual ou menor que b.

Mas o objetivo principal desse código **não é apenas comparar os números, e sim ensinar uma regra de ouro do VHDL**: como evitar a criação acidental de memória (chamada de *Latch*).

1. O Início (Bibliotecas e Entidade)

- **Bibliotecas:** Além do padrão, ele usa a `numeric_std` porque precisa tratar os vetores lógicos como números matemáticos reais com sinal positivo/negativo (`signed`).
- **A Entidade:** O chip tem duas entradas de 8 bits (a e b) e três saídas de 1 bit: `gt` (*greater than* / maior que), `eq` (*equal* / igual) e `lt` (*less than* / menor que).

2. O Problema (A Arquitetura Incorreta)

- **atribuicao_incorreta:** O código usa um bloco process com if/else para testar os números.
- **O Erro:** Quando ele descobre que $a > b$, ele manda $gt \leq '1'$. Mas ele **esquece de desligar** os outros pinos (eq e lt).
- **A Consequência:** Em linguagens como C ou Python, isso não seria um problema. Mas em VHDL (que é hardware físico), se você não disser explicitamente o que acontece com uma saída em todas as situações, o chip entende que deve "memorizar" o valor anterior. Isso cria um circuito de memória indesejado (*Latch*), que gasta espaço e causa falhas terríveis de temporização.

3. As Soluções (As Arquiteturas Corretas)

O professor mostra duas formas de corrigir esse erro grave:

- **atribuicao_correta_v1 (Força Bruta):** Aqui, o programador escreve o que acontece com **todas as três saídas** dentro de cada bloco do if, do elsif e do else. Se um pino liga, ele garante no código que os outros dois recebem '0'. O código fica maior, mas o chip é construído corretamente, sem memória.
- **atribuicao_correta_v2 (A mais elegante / Boas Práticas):** Esta é a forma mais inteligente. Logo no início do process, **antes** do if, o código atribui '0' para todas as saídas ($gt \leq '0'$; $eq \leq '0'$; $lt \leq '0'$;). Como comandos dentro de um process são lidos de forma sequencial (de cima para baixo), as saídas já começam todas desligadas por padrão. O if que vier na sequência só precisa acender a saída correta sobrescrevendo o '0' por um '1'. Zero risco de criar *Latches* e o código fica muito mais limpo.

4. A Configuração (configuration)

- Assim como no multiplexador, a configuração no final do arquivo atua como um menu. Repare que as duas primeiras opções estão comentadas com `--`. O programador está dizendo ao compilador: *"Grave o chip usando a arquitetura atribuicao_correta_v2"*.

ATRIBUIÇÃO SEQUENCIAL .DO V1

```
vlib rtl_work
```

```
vmap work rtl_work
```

```
vmap work rtl_work
```

```
vcom -93 -work work {/home/hiago.s2006/eld-II/aula-28-04-2026/atribuicao_sequencial.vhd}
```

```
vsim work.atribuicao_sequencial(atribuicao_incorreta)
```

```
add wave -position insertpoint \
```

```
sim:/atribuicao_sequencial/a \
```

```
sim:/atribuicao_sequencial/b \
```

```
sim:/atribuicao_sequencial/gt \  
sim:/atribuicao_sequencial/eq \  
sim:/atribuicao_sequencial/lt  
force -freeze sim:/atribuicao_sequencial/a 00000001 0  
force -freeze sim:/atribuicao_sequencial/b 00000000 0  
run  
  
force -freeze sim:/atribuicao_sequencial/b 00000001 0  
run  
  
force -freeze sim:/atribuicao_sequencial/b 00000010 0  
run  
  
force -freeze sim:/atribuicao_sequencial/a 10000000 0  
force -freeze sim:/atribuicao_sequencial/b 00000001 0  
run  
  
force -freeze sim:/atribuicao_sequencial/a 11111111 0  
force -freeze sim:/atribuicao_sequencial/b 10000000 0  
run  
  
force -freeze sim:/atribuicao_sequencial/a 11111011 0  
force -freeze sim:/atribuicao_sequencial/b 11111011 0  
run  
  
force -freeze sim:/atribuicao_sequencial/a 11111111 0  
force -freeze sim:/atribuicao_sequencial/b 00000000 0  
run  
  
force -freeze sim:/atribuicao_sequencial/a 00000000 0  
force -freeze sim:/atribuicao_sequencial/b 11111111 0  
run
```

ATRIBUIÇÃO CORRETA .DO V2

```
vlib rtl_work

vmap work rtl_work


vcom -93 -work work {/home/hiago.s2006/eld-II/aula-28-04-2026/atribuicao_sequencial.vhd}


vsim work.atribuicao_sequencial(atribuicao_correta_v2)


add wave -position insertpoint \
sim:/atribuicao_sequencial/a \
sim:/atribuicao_sequencial/b \
sim:/atribuicao_sequencial/gt \
sim:/atribuicao_sequencial/eq \
sim:/atribuicao_sequencial/lt

force -freeze sim:/atribuicao_sequencial/a 10000000 0

force -freeze sim:/atribuicao_sequencial/b 10000000 0

run

force -freeze sim:/atribuicao_sequencial/b 00000001 0

run

force -freeze sim:/atribuicao_sequencial/a 11111111 0

force -freeze sim:/atribuicao_sequencial/b 10000000 0

run

force -freeze sim:/atribuicao_sequencial/a 11111011 0

force -freeze sim:/atribuicao_sequencial/b 11111011 0

run

force -freeze sim:/atribuicao_sequencial/a 11111111 0

force -freeze sim:/atribuicao_sequencial/b 00000000 0

run

force -freeze sim:/atribuicao_sequencial/a 00000000 0
```

```
force -freeze sim:/atribuicao_sequencial/b 11111111 0
```

```
run
```

ATRIBUIÇÃO CORRETA . DO

```
vlib rtl_work
```

```
vmap work rtl_work
```

```
vmap work rtl_work
```

```
vcom -93 -work work {/home/hiago.s2006/eld-II/aula-28-04-2026/atribuicao_sequencial.vhd}
```

```
vsim work.atribuicao_sequencial(atribuicao_incorreta)
```

```
add wave -position insertpoint \
```

```
sim:/atribuicao_sequencial/a \
```

```
sim:/atribuicao_sequencial/b \
```

```
sim:/atribuicao_sequencial/gt \
```

```
sim:/atribuicao_sequencial/eq \
```

```
sim:/atribuicao_sequencial/lt
```

```
force -freeze sim:/atribuicao_sequencial/a 00000001 0
```

```
force -freeze sim:/atribuicao_sequencial/b 00000000 0
```

```
run
```

```
force -freeze sim:/atribuicao_sequencial/b 00000001 0
```

```
run
```

```
force -freeze sim:/atribuicao_sequencial/b 00000010 0
```

```
run
```

```
force -freeze sim:/atribuicao_sequencial/a 10000000 0
```

```
force -freeze sim:/atribuicao_sequencial/b 00000001 0
```

```
run
```

```
force -freeze sim:/atribuicao_sequencial/a 11111111 0
force -freeze sim:/atribuicao_sequencial/b 10000000 0
run
```

```
force -freeze sim:/atribuicao_sequencial/a 11111011 0
force -freeze sim:/atribuicao_sequencial/b 11111011 0
run
```

```
force -freeze sim:/atribuicao_sequencial/a 11111111 0
force -freeze sim:/atribuicao_sequencial/b 00000000 0
run
```

```
force -freeze sim:/atribuicao_sequencial/a 00000000 0
force -freeze sim:/atribuicao_sequencial/b 11111111 0
run
```

CÓDIGO 3

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity bcd2dsp is
    port
    (
        sw : in std_logic_vector(3 downto 0);
        dsp : out std_logic_vector(6 downto 0)
    );
end bcd2dsp;

architecture implementacao of bcd2dsp is
begin
    process(sw)
    begin
        case sw is
            when "0000" => dsp <= "0000001";
```

```

        when "0001" => dsp <= "1001111";
        when "0010" => dsp <= "0010010";
        when "0011" => dsp <= "0000110";
        when "0100" => dsp <= "1001100";
        when "0101" => dsp <= "0100100";
        when "0110" => dsp <= "0100000";
        when "0111" => dsp <= "0001111";
        when "1000" => dsp <= "0000000";
        when "1001" => dsp <= "0001100";
        when others => dsp <= "1111111";
    end case;
end process;
end implementacao;

configuration cfg of bcd2dsp is
    for implementacao end for;
end cfg;

```

Esse código descreve um **Decodificador BCD para Display de 7 Segmentos**. Ele traduz um número binário de 4 bits (de 0 a 9) para o padrão visual exato que acende as luzes de um visor digital.

A Entidade (port): A entrada sw (switches/chaves) recebe um número de 4 bits. A saída dsp tem 7 bits, onde cada bit representa um "tracinho" de luz do display (os segmentos a, b, c, d, e, f, g).

A Pegadinha (Anodo Comum): Repare bem nos números! Para mostrar o zero (when "0000"), a saída é "0000001". Isso nos diz que esse display funciona em lógica invertida (**Anodo Comum**): o painel **acende quando recebe '0' e apaga quando recebe '1'**. O traço do meio (segmento 'g', o último bit) recebe '1' para ficar apagado, formando assim o número "0" no visor.

A Estrutura (case): O bloco usa a estrutura case para analisar a entrada. É a forma mais limpa e eficiente em VHDL para fazer uma "tabela de tradução" fixa, sendo lida linha por linha.

A Importância do others: Um vetor de 4 bits pode contar de 0 a 15, mas o sistema BCD (*Binary-Coded Decimal*) só se importa com os números de 0 a 9. O comando when others => dsp <= "1111111" serve para duas coisas cruciais:

1. Garantir que as opções inválidas (de 10 a 15) deixem o display totalmente apagado (todos recebendo '1').
2. Seguir a regra de ouro do VHDL e mapear todas as possibilidades, evitando a criação acidental de memória (*Latch*).

CÓDIGO 4

-- Implementar um Flip Flop sensível à borda de subida com reset assíncrono e enable.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

```

```

entity FlipFlopComEnable is
    port

```

```

        (
            clk: in std_logic;
            d: in std_logic;
            q: out std_logic;
            rst: in std_logic;
            enable: in std_logic
        );
end FlipFlopComEnable;

-- IMPLEMENTACAO DE FLIPFLOP D SENSIVEL A BORDA DE SUBIDA COM ENABLE E
RESET ASSINCRONO
-- ESSA NAO Eh UMA IMPLEMENTACAO CORRETA PARA A DISCIPLINA
architecture implementacao_v1 of FlipFlopComEnable is
    signal estadoAtual : std_logic;
begin
    process(clk, rst)
    begin
        if rst = '1' then
            estadoAtual <= '0';
        elsif rising_edge(clk) then
            if enable = '1' then
                estadoAtual <= d;
            else
                estadoAtual <= estadoAtual;
            end if;
        end if;
    end process;

    q <= estadoAtual;

end implementacao_v1;

configuration cfg of FlipFlopComEnable is
    for implementacao_v1 end for;
end cfg;

```

Esse código descreve um **Flip-Flop tipo D com Enable e Reset Assíncrono**. O Flip-Flop é a unidade mais básica de memória digital, responsável por "lembrar" de 1 bit de informação.

A Entidade (port): Temos o sinal do relógio (clk), o dado de entrada (d), o botão de emergência para zerar a memória (rst), a chave que autoriza a gravação (enable) e a saída com o valor salvo (q).

Reset Assíncrono: Repare no process(clk, rst). Como o rst está nessa lista é a primeira coisa a ser testada no código (if rst = '1'), isso significa que se o sinal de reset for acionado, a saída vai para zero **imediatamente**, independentemente do que o relógio (clk) estiver fazendo.

Sincronismo (rising_edge): Se não tiver reset, o circuito espera a borda de subida do relógio (quando o sinal vai de 0 para 1). Se nesse exato instante o enable estiver ligado ('1'), ele copia o valor de d para a memória.

A Falha (O motivo do aviso no código): O professor escreveu que *"ESSA NÃO É UMA IMPLEMENTAÇÃO CORRETA PARA A DISCIPLINA"*. O erro conceitual está na linha `else estadoAtual <= estadoAtual;`. Em VHDL, dentro de um bloco de relógio, se você não manda a variável mudar, o hardware **automaticamente** mantém o valor anterior (é para isso que serve um Flip-Flop!). Dizer para o circuito que "o estado atual deve receber o estado atual" é uma redundância desnecessária. Para corrigir, bastaria apagar todo esse trecho do `else`.

```
. DO

vlib rtl_work
vmap work rtl_work
vmap work rtl_work

vcom -93 -work work
{/home/hiago.s2006/eld-II/aula-07-05-2026/FlipFlopComEnable.vhd}

vsim rtl_work.flipflopcomenable(implementacao_v1)

add wave -position insertpoint \
sim:/flipflopcomenable/clk \
sim:/flipflopcomenable/d \
sim:/flipflopcomenable/q \
sim:/flipflopcomenable/rst \
sim:/flipflopcomenable/enable \
sim:/flipflopcomenable/estadoAtual

force -freeze sim:/flipflopcomenable/enable 1 0
force -freeze sim:/flipflopcomenable/rst 1 0
force -freeze sim:/flipflopcomenable/clk 0 0
force -freeze sim:/flipflopcomenable/d 0 0
run
force -freeze sim:/flipflopcomenable/rst 0 0
force -freeze sim:/flipflopcomenable/clk 1 0
force -freeze sim:/flipflopcomenable/d 1 0
run
force -freeze sim:/flipflopcomenable/clk 0 0
force -freeze sim:/flipflopcomenable/d 0 0
run
force -freeze sim:/flipflopcomenable/clk 1 0
run
force -freeze sim:/flipflopcomenable/enable 0 0
force -freeze sim:/flipflopcomenable/clk 0 0
force -freeze sim:/flipflopcomenable/d 1 0
run
force -freeze sim:/flipflopcomenable/clk 1 0
run
force -freeze sim:/flipflopcomenable/rst 1 0
run
```

CÓDIGO 5

-- BINARY COUNTER WITH MAX PULSE IMPLEMENTATION

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ContadorBinario is
    port
    (
        clk,reset: in std_logic;
        max_pulse: out std_logic;
        q: out std_logic_vector(3 downto 0)
    );
end ContadorBinario;

```

```

architecture exemplo_v1 of ContadorBinario is
    signal r_reg: unsigned(3 downto 0);
    signal r_next: unsigned(3 downto 0);
begin
    -- register
    process(clk, reset)
    begin
        if (reset = '1') then
            r_reg <= (others => '0');
        elsif (clk'event and clk = '1') then
            r_reg <= r_next;
        end if;
    end process;

    -- next-state logic
    r_next <= r_reg + 1;

    -- output logic
    q <= std_logic_vector(r_reg);

    max_pulse <= '1' when r_reg = "1111" else '0';

end exemplo_v1;

```

Esse código descreve um **Contador Binário de 4 bits com Pulso de Máximo**. Basicamente, é um circuito que conta de 0 a 15 em loop, no ritmo do relógio, e avisa quando atinge o valor limite.

A Entidade (port): O chip recebe o sinal do relógio (clk) e o botão de zerar (reset). Como saídas, ele entrega o número atual da contagem em q (um vetor de 4 bits) e um sinal de alerta de "capacidade máxima" chamado max_pulse.

A Estrutura Profissional: Este código usa uma ótima prática de design digital, dividindo o hardware em três blocos bem definidos:

- **A Memória / Registrador (process):** É o bloco acionado pelo relógio e pelo reset. Se o reset for acionado, a memória zera instantaneamente (others => '0'). Se não, a cada pulso do relógio (na borda de subida: clk'event and clk = '1'), a memória atualiza para o próximo número.
- **A Lógica do Próximo Estado (r_next):** É o "matemático" do circuito. Ele funciona o tempo todo pegando o número atual da memória e somando 1 (r_reg + 1).
- **A Lógica de Saída (q e max_pulse):** Conecta a memória às "pernas" do chip (q <= std_logic_vector(r_reg)). A sacada principal é o max_pulse: ele age como uma bandeira que levanta ('1') estritamente no momento em que a memória atinge "1111" (o número 15 em decimal). Quando o relógio bater novamente e o contador zerar para começar tudo de novo, a bandeira abaixa ('0').

. DO

```
vlib rtl_work
```

```
vmap work rtl_work
```

```
vmap work rtl_work
```

```
vcom -93 -work work {/home/hiago.s2006/eld-II/aula-12-05-2026/ContadorDecimal.vhd}
```

```
vsim work.contadordecimal(two_seg_arch)
```

```
add wave -position insertpoint \
```

```
sim:/contadordecimal/clk \
```

```
sim:/contadordecimal/reset \
```

```
sim:/contadordecimal/q \
```

```
sim:/contadordecimal/r_reg \
```

```
sim:/contadordecimal/r_next
```

```
vlib rtl_work
```

```
vmap work rtl_work
```

```
vmap work rtl_work
```

```
vcom -93 -work work {/home/hiago.s2006/eld-II/aula-12-05-2026/ContadorDecimal.vhd}
```

```
vsim work.contadordecimal(two_seg_arch)
```

```
add wave -position insertpoint \
```

```
sim:/contadordecimal/clk \
```

```
sim:/contadordecimal/reset \
```

```
sim:/contadordecimal/q \
```

```
sim:/contadordecimal/r_reg \
```

```
sim:/contadordecimal/r_next
```

```
force -freeze sim:/contadordecimal/reset 1 0
```

```
force -freeze sim:/contadordecimal/clk 1 0
```

```
run
```

```
force -freeze sim:/contadordecimal/clk 0 0
```

```
force -freeze sim:/contadordecimal/reset 0 0
```

```
run
```

```
force -freeze sim:/contadordecimal/clk 1 0
```

```
run
```

```
force -freeze sim:/contadordecimal/clk 0 0
```

```
run
```

```
force -freeze sim:/contadordecimal/clk 1 0
```

```
run
```

```
force -freeze sim:/contadordecimal/clk 0 0
```

```
run
```

```
force -freeze sim:/contadordecimal/clk 1 0
```

```
run
```

```
force -freeze sim:/contadordecimal/clk 0 0
```

```
run
```

[illegible]

```

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

```

CÓDIGO 6

-- MOD10 COUNTER IMPLEMENTATION

```

library ieee;

use ieee.std_logic_1164.all;

use ieee.numeric_std.all;

entity ContadorDecimal is

    port(

```

```

    clk, reset : in std_logic;

    q          : out std_logic_vector(3 downto 0)

);

end ContadorDecimal;

architecture two_seg_arch of ContadorDecimal is

    constant TEN : integer := 10;

    signal r_reg : unsigned(3 downto 0);

    signal r_next : unsigned(3 downto 0);

begin

    -- register

    process(clk, reset)

    begin

        if (reset = '1') then

            r_reg <= (others => '0');

        elsif (clk'event and clk = '1') then

            r_reg <= r_next;

        end if;

    end process;

    -- next-state logic

    r_next <= (others => '0') when r_reg = (TEN - 1) else

        r_reg + 1;

    -- output logic

    q <= std_logic_vector(r_reg);

end two_seg_arch;

```

Esse código descreve um **Contador Decimal (Módulo-10)**. Diferente do contador anterior que ia até 15, este aqui é feito para contar apenas de **0 a 9** (como os nossos números decimais) e depois reiniciar.

- **A Entidade (port):** Igual ao anterior. Recebe relógio (clk), botão de emergência (reset) e entrega a contagem em 4 bits (q).
- **A Constante:** Ele cria uma constante matemática no início chamada TEN com valor 10. Isso deixa o código mais legível para quem for ler no futuro.
- **A Estrutura (Boas Práticas):** Mantém a divisão profissional em 3 partes: a memória (o process com o relógio), a lógica matemática (r_next) e a saída (q).
- **A Grande Sacada (Lógica de Próximo Estado):** É aqui que a mágica do "Módulo-10" acontece. A linha `r_next <= (others => '0') when r_reg = (TEN - 1) else r_reg + 1;` dita a seguinte regra:
 - O circuito olha para a memória atual. Se o número for **9** (que é `TEN - 1`), ele avisa que o próximo número deve ser **zero** (`others => '0'`).
 - **Senão** (else), ele simplesmente pega o número atual e soma 1.

Resumindo: Ele conta 0, 1, 2... até o 9. Quando bate no 9, o código intercepta a conta e força a voltar pro 0 no próximo pulso do relógio.

. DO

```
vlib rtl_work
```

```
vmap work rtl_work
```

```
vmap work rtl_work
```

```
vcom -93 -work work
{/home/hiago.s2006/eld-II/aula-12-05-2026/ContadorDecimal.vhd}
```

```
vsim work.contadordecimal(two_seg_arch)
```

```
add wave -position insertpoint \
```

```
sim:/contadordecimal/clk \
```

```
sim:/contadordecimal/reset \
```

```
sim:/contadordecimal/q \
```

```
sim:/contadordecimal/r_reg \
```

```
sim:/contadordecimal/r_next
```

```
vlib rtl_work
```

```
vmap work rtl_work
```

```
vmap work rtl_work
```

```
vcom -93 -work work  
{/home/hiago.s2006/eld-II/aula-12-05-2026/ContadorDecimal.vhd}
```

```
vsim work.contadordecimal(two_seg_arch)
```

```
add wave -position insertpoint \
```

```
sim:/contadordecimal/clk \
```

```
sim:/contadordecimal/reset \
```

```
sim:/contadordecimal/q \
```

```
sim:/contadordecimal/r_reg \
```

```
sim:/contadordecimal/r_next
```

```
force -freeze sim:/contadordecimal/reset 1 0
```

```
force -freeze sim:/contadordecimal/clk 1 0
```

```
run
```

```
force -freeze sim:/contadordecimal/clk 0 0
```

```
force -freeze sim:/contadordecimal/reset 0 0
```

```
run
```

```
force -freeze sim:/contadordecimal/clk 1 0
```

```
run
```

```
force -freeze sim:/contadordecimal/clk 0 0
```

```
run
```

```
force -freeze sim:/contadordecimal/clk 1 0
```

```
run
```

```
force -freeze sim:/contadordecimal/clk 0 0
```

```
run
```

[illegible]

```
run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run
```

CÓDIGO 7

```
library ieee;

use ieee.std_logic_1164.all;

entity ShiftRightRegister is
```

```

port(
    clk, reset : in std_logic;

    d          : in std_logic;

    q          : out std_logic
);

end ShiftRightRegister;

architecture two_seg_arch of ShiftRightRegister is

    signal r_reg : std_logic_vector(3 downto 0);

    signal r_next : std_logic_vector(3 downto 0);

begin

    -- register
    process(clk, reset)
    begin
        if (reset = '1') then
            r_reg <= (others => '0');

        elsif (clk'event and clk = '1') then
            r_reg <= r_next;
        end if;
    end process;

    -- next-state logic (shift right 1 bit)
    r_next <= d & r_reg(3 downto 1);

```

```
-- output
```

```
q <= r_reg(0);
```

```
end two_seg_arch;
```

Esse código descreve um **Registrador de Deslocamento para a Direita** (*Shift Right Register*). Imagine um tubo onde você insere uma bolinha (bit) de informação por vez, e a cada pulso do relógio, todas as bolinhas que já estão lá dentro andam uma casa para a direita.

A Entidade (port): O circuito recebe o relógio (clk), o botão de limpar memória (reset) e um único bit de entrada por vez (d). Ele devolve um único bit de saída por vez (q).

A Capacidade Oculta (Os Sinais): Embora o chip só tenha 1 pino de entrada e 1 de saída de dados, as variáveis internas r_reg e r_next são vetores de 4 posições (3 downto 0). Isso significa que o chip consegue armazenar até 4 bits em fila lá dentro.

O Bloco de Memória (process): Segue a regra padrão profissional. O reset apaga a fila inteira. Se não houver reset, a memória avança para o próximo estado a cada subida do relógio.

A Lógica de Deslocamento (r_next): A linha de ouro é r_next <= d & r_reg(3 downto 1);. O símbolo & serve para concatenar (colar) informações. O circuito pega o novo bit da porta d e coloca na primeira posição da fila (à esquerda), empurrando os 3 primeiros bits da fila atual (r_reg de 3 a 1) para a direita. O bit que estava na última posição da direita simplesmente "cai fora" do vetor do próximo estado.

A Lógica de Saída (q): A linha q <= r_reg(0); liga o pino de saída do chip permanentemente à última casa da direita da fila. Assim, o bit que foi empurrado até o final do corredor é finalmente entregue ao mundo exterior.

. DO

```
vlib rtl_work
```

```
vmap work rtl_work
```

```
vcom -93 -work work {/home/hiago.s2006/eld-II/aula-12-05-2026/ShiftRightRegister.vhd}
```

```
vsim work.shiftrightregister(two_seg_arch)
```

```
add wave -position insertpoint \
```

```
sim:/shiftrightregister/clk \
```

```
sim:/shiftrightregister/reset \
```

sim:/shiftrightregister/d \

sim:/shiftrightregister/q \

sim:/shiftrightregister/r_reg \

sim:/shiftrightregister/r_next

force -freeze sim:/shiftrightregister/reset 1 0

force -freeze sim:/shiftrightregister/clk 1 0

force -freeze sim:/shiftrightregister/d 1 0

run

force -freeze sim:/shiftrightregister/reset 0 0

force -freeze sim:/shiftrightregister/clk 0 0

run

force -freeze sim:/shiftrightregister/clk 1 0

run

force -freeze sim:/shiftrightregister/clk 0 0

run

force -freeze sim:/shiftrightregister/clk 1 0

run

force -freeze sim:/shiftrightregister/d 0 0

force -freeze sim:/shiftrightregister/clk 0 0

run

force -freeze sim:/shiftrightregister/clk 1 0

run

force -freeze sim:/shiftrightregister/clk 0 0

run

force -freeze sim:/shiftrightregister/clk 1 0

run

	Engenharia de Telecomunicações Eletrônica Digital II	Prof. Roberto de Matos roberto.matos@ifsc.edu.br
Circuitos Sequenciais com VHDL		

1. Implemente um registrador de deslocamento universal que pode carregar uma palavra de dados paralela, realizar deslocamento em qualquer direção ou pausar. Um sinal de controle, `ctrl`, especifica a operação desejada: pausar, deslocar para a esquerda, deslocar para a direita e carregar.
2. Sabendo que um contador sequencial percorre uma sequência predefinida de estados e que é a lógica do próximo estado que determina os padrões na sequência, implemente um contador que percorre a seguinte sequência arbitrária "000", "011", "100", "101" e "111".
3. Considere um contador binário de 8 bits, com contagem crescente e decrescente. Ele possui um sinal de controle, `up`, incrementando quando o sinal `up` é '1' e decrementando caso contrário. Desenhe o diagrama conceitual seguindo os conceitos de projeto síncrono e desenvolva o código VHDL para este circuito.

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.NUMERIC_STD.ALL;
```

```
entity registradorUniversal is
```

```
    port
```

```
    (
```

```
        clk, reset: in STD_LOGIC;
```

```
        d: in STD_LOGIC;
```

```
        control: in STD_LOGIC_VECTOR(1 downto 0); -- apenas 4 opções de ação
```

```
        d_to_right: in STD_LOGIC;
```

```
        d_to_left: in STD_LOGIC;
```

```
        palavra: in STD_LOGIC_VECTOR(3 downto 0); -- palavra de 3 bits
```

```
        q: out STD_LOGIC_VECTOR(3 downto 0)
```

```
);
end registradorUniversal;
```

```
-- 00 para
-- 10 deslocar esquerda
-- 01 deslocar direita
-- 11 pausar
```

```
architecture ex_01 of registradorUniversal is
    signal r_reg : std_logic_vector(3 downto 0);
    signal r_next : std_logic_vector(3 downto 0);
begin
```

```
    -- register
    process(clk, reset)
    begin
        if (reset = '1') then
            r_reg <= (others => '0');

            elsif (clk'event and clk = '1') then
                r_reg <= r_next;
            end if;
        end process;
```

```
    process (control, r_reg, palavra, d_to_left, d_to_right)
    begin
        case control is
            when "00" => r_next <= r_reg;
            when "01" => r_next <= d_to_right & r_reg(3 downto 1); -- concatena para direita
            when "10" => r_next <= r_reg(2 downto 0) & d_to_left; -- concatena para a esquerda
            when "11" => r_next <= palavra;
            when others => r_next <= r_reg;
        end case;
    end process;
```

```
    q <= r_reg;
end ex_01;
```

```
configuration ex_01 of registradorUniversal is
    for ex_01 end for;
end configuration;
```

YAGO: A ideia central do exercício é ter um flip flop D que registre a operação de um bit shift tanto para a esquerda quanto para a direita, também é necessário que tenha controle sobre para qual lado será o shift e qual a palavra, ou seja, sequência de bits que irá ser usada para fazer a mudança

Importante notar que em VHDL o operador "&" faz essa operação de rotear o bit ao final o ao início do argumento que foi passado, ou seja, tem que analisar como que está o vetor de bits para posicionar o bit novo na posição que desejamos

Exemplo:

CONCATENAÇÃO PARA A ESQUERDA

Linha do código: `r_next <= d_to_right & r_reg(3 downto 1);`

Valores do teste:

`d_to_right = 1`

`r_reg = 1010` --valor registrado anteriormente e está em memória

Passo 1: O código olha para a fatia `r_reg(3 downto 1)`.

No vetor 1010, os bits nas posições 3, 2 e 1 são os três primeiros bits.

`r_reg(3)` é 1

`r_reg(2)` é 0

`r_reg(1)` é 1

Portanto, `r_reg(3 downto 1)` isola o valor: 101

Passo 2: O código faz a concatenação usando o `&`.

Ele junta `d_to_right` com a fatia isolada:

`r_next <= 1 & 101;`

Passo 3: Na próxima subida do clock, `r_reg` recebe o valor de `r_next`.

Resultado final de `r_reg`: 1101

CONCATENAÇÃO PARA A DIREITA

Linha do código: `r_next <= r_reg(2 downto 0) & d_to_left;`

Valores do teste:

`d_to_left = 0`

`r_reg = 1010`

Passo 1: O código olha para a fatia `r_reg(2 downto 0)`.

No vetor 1010, os bits nas posições 2, 1 e 0 são os três últimos bits.

`r_reg(2)` é 0

`r_reg(1)` é 1

`r_reg(0)` é 0

Portanto, `r_reg(2 downto 0)` isola o valor: 010

Passo 2: O código faz a concatenação usando o `&`.

Ele junta a fatia isolada com o `d_to_left` no final:

`r_next <= 010 & 0;`

Passo 3: Na próxima subida do clock, `r_reg` recebe o valor de `r_next`.

Resultado final de `r_reg`: 0100

GEMINI: Esse código descreve um **Registrador de Deslocamento Universal de 4 bits**. Ele é considerado "universal" porque combina várias funções de memória em um único circuito: ele pode manter os dados intactos, deslocar bits para a direita, deslocar bits para a esquerda ou carregar uma informação inteira de uma só vez.

Aqui está o resumo direto do funcionamento (com um aviso sobre os comentários do seu professor):

- **A Entidade (port):** Além do relógio e do reset, o pino control (2 bits) age como um "controle remoto" que escolhe a função do momento. O chip tem entradas separadas para inserir bits quando desloca para a direita (d_to_right) ou para a esquerda (d_to_left). Também possui uma entrada paralela de 4 bits (palavra).
 - Correção 1: O pino d declarado na entidade não está sendo usado em lugar nenhum do código.
 - Correção 2: O comentário do professor diz "palavra de 3 bits", mas o vetor 3 downto 0 significa que são **4 bits**.
- **A Estrutura Profissional:** Continua usando as melhores práticas. O primeiro bloco process lida estritamente com a atualização da memória (relógio e reset). O segundo bloco process atua como um roteador puramente lógico que decide qual será o próximo valor (r_next).
- **As 4 Operações (case control):** É aqui que a mágica acontece. *Atenção, pois os comentários do código original estão trocados em relação ao que o código realmente faz:*
 - "00" (**Manter**): r_next <= r_reg. O chip congela e apenas mantém a memória atual.
 - "01" (**Deslocar Direita**): Insere o bit de d_to_right na esquerda e empurra o restante do vetor para a direita.
 - "10" (**Deslocar Esquerda**): Empurra o vetor para a esquerda e insere o novo bit de d_to_left na ponta direita.
 - "11" (**Carga Paralela**): *O comentário do código diz "pausar", mas isso é um erro.* A linha r_next <= palavra faz um "Load" paralelo. O chip ignora deslocamentos de fila e injeta os 4 bits da porta palavra diretamente na memória de uma só vez.
- **A Saída (q):** O vetor q reflete constantemente o estado atual dos 4 bits que estão salvos na memória (r_reg).

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use IEEE.NUMERIC_STD.ALL;
```

```
entity contadorSequencial is
```

```
    port
```

```
    (
```

```
        clk, rst: in STD_LOGIC;
```

```
        q: out STD_LOGIC_VECTOR(2 downto 0)
```

```
    );
```

```
end contadorSequencial;
```

```
architecture ex_02 of contadorSequencial is
```

```
    signal q_contador : STD_LOGIC_VECTOR(2 downto 0);
```

```
    signal d_contador : STD_LOGIC_VECTOR(2 downto 0);
```

```
begin
```

```
    process (clk,rst)
```

```
    begin
```

```
        if (rst = '1') then
```

```
            q_contador <= "000";
```

```
        elsif (rising_edge(clk)) then
```

```
            q_contador <= d_contador;
```

```

        end if;
    end process;

    process (q_contador)
    begin
        case q_contador is
            when "000" => d_contador <= "011";
            when "011" => d_contador <= "100";
            when "100" => d_contador <= "101";
            when "101" => d_contador <= "111";
            when others => d_contador <= "000";
        end case;
    end process;

    q <= q_contador;

end ex_02;

configuration cfg of contadorSequencial is
    for ex_02 end for;
end;

```

YAGO: Entendendo o Contador Sequencial Arbitrário em VHDL

Sabendo que um contador sequencial percorre uma sequência predefinida de estados e que é a lógica do próximo estado que determina os padrões na sequência, implemente um contador que percorre a seguinte a sequência arbitrária "000", "011", "100", "101" e "111".

1. O Propósito do Exercício

O objetivo principal deste projeto é demonstrar como construir uma Máquina de Estados Síncrona. Ou seja, com sinal de clock e lógica de registro, próximo estado e output.

Neste exercício, a sequência exigida foi arbitrária: "000" → "011" → "100" → "101" → "111".

Para que o hardware execute esse salto customizado, o projeto foi dividido em duas funções principais de hardware:

1. Memória (Os Flip-Flops): Guarda o número que está aceso na tela no momento presente. (Lógica de registro)
2. Lógica Combinacional (As Portas Lógicas): Fica lendo o número presente e calcula "matematicamente" qual deve ser o próximo número da lista. (Lógica do próximo estado)

2. A Diferença entre Entidade e Arquitetura no VHDL

Ao programar em VHDL, nós não estamos escrevendo software; estamos descrevendo um chip físico. Por isso, a linguagem divide o código em duas partes estritas: a Entidade (entity) e a Arquitetura (architecture).

A Entidade:

A entity descreve apenas a parte externa do chip que estamos disposição

- Entradas (clk, rst): São pinos que recebem sinais de entrada
- Saídas (q): O pino que enviam sinais de saída

O Problema do pino de saída (out): No VHDL, um pino declarado como out funciona como uma rua de mão única para fora do chip. O circuito interno não consegue "ler"

o que está passando por esse pino. Por isso declaramos sinais dentro da arquitetura.

A Arquitetura

A architecture descreve tudo o que acontece dentro do chip que estamos implementando. É aqui que declaramos os nossos Sinais Internos (signals). Esses sinais em si não são pinos externos que estão expostos para o uso, isso é papel da entidade, mas são usados internamente na implementação do circuito interno.

3. O Papel dos Sinais Internos: q vs q_contador vs d_contador

Como o nosso circuito precisa "olhar" para o valor atual para conseguir prever o futuro (qual é o próximo número da sequência), nós não podemos usar a porta externa q que é a saída do chip, pois ela não pode ser lida internamente, podemos dizer qual valor ela irá assumir para a saída, mas não tem como ler ela de dentro do circuito.

A solução é criar fios internos para fazer o circuito funcionar, fazer com que tenha o fio de retorno, igual fazemos nos desenhos a mão com portas lógicas e multiplexadores:

- q_contador (O Presente): É o fio interno que guarda a saída atual dos flip-flops. Toda a lógica de controle pode ler este fio sem problemas para saber em qual estado o contador está.
- d_contador (O Futuro): É o fio interno que carrega o resultado do cálculo. Quando o circuito decide qual é o próximo estado, ele coloca esse valor em d_contador e o deixa estacionado na entrada dos flip-flops, esperando o pulso de relógio (clk). Ou seja, esse sinal é resultado de q (estado presente naquele clock) ter passado pela lógica do próximo estado gerando o valor d, que no próximo clock será o próximo q e assim sucessivamente.

O que conecta essa implementação interna com o exterior do chip

```
q <= q_contador;
```

Essa parte do código que conecta nossa lógica do próximo estado com a saída q declarada na entidade.

NOVO CÓDIGO

```
--=====
```

```
-- Listing 9.5 timer
```

```
--=====
```

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity timer is
  port(
    clk, reset: in std_logic;
    sec,min: out std_logic_vector(5 downto 0)
  );
end timer;

architecture multi_clock_arch of timer is
  signal r_reg: unsigned(25 downto 0);
  signal r_next: unsigned(25 downto 0);
  signal s_reg, m_reg: unsigned(5 downto 0);
  signal s_next, m_next: unsigned(5 downto 0);
  signal sclk, mclk: std_logic;
```

```

begin
  -- register
  process(clk,reset)
  begin
    if (reset='1') then
      r_reg <= (others=>'0');
    elsif (clk'event and clk='1') then
      r_reg <= r_next;
    end if;
  end process;
  -- next-state logic
  r_next <= (others=>'0') when r_reg=49999999 else
    r_reg + 1;
  -- output logic
  sclk <= '0' when r_reg < 25000000 else
    '1';
  -- second divider
  process(sclk,reset)
  begin
    if (reset='1') then
      s_reg <= (others=>'0');
    elsif (sclk'event and sclk='1') then
      s_reg <= s_next;
    end if;
  end process;
  -- next-state logic
  s_next <= (others=>'0') when s_reg=59 else
    s_reg + 1;
  -- output logic
  mclk <= '0' when s_reg < 30 else
    '1';
  sec <= std_logic_vector(s_reg);
  -- minute divider
  process(mclk,reset)
  begin
    if (reset='1') then
      m_reg <= (others=>'0');
    elsif (mclk'event and mclk='1') then
      m_reg <= m_next;
    end if;
  end process;
  -- next-state logic
  m_next <= (others=>'0') when m_reg=59 else
    m_reg + 1;
  -- output logic
  min <= std_logic_vector(m_reg);
end multi_clock_arch;

```

```

=====
-- Listing 9.6
=====
architecture single_clock_arch of timer is
  signal r_reg: unsigned(25 downto 0);
  signal r_next: unsigned(25 downto 0);
  signal s_reg, m_reg: unsigned(5 downto 0);
  signal s_next, m_next: unsigned(5 downto 0);
  signal s_en, m_en: std_logic;
begin
  -- register
  process(clk,reset)
  begin
    if (reset='1') then
      r_reg <= (others=>'0');
      s_reg <= (others=>'0');
      m_reg <= (others=>'0');
    elsif (clk'event and clk='1') then
      r_reg <= r_next;
      s_reg <= s_next;
      m_reg <= m_next;
    end if;
  end process;
  -- next-state logic/output logic for mod-50000000 counter
  r_next <= (others=>'0') when r_reg=49999999 else
    r_reg + 1;
  s_en <= '1' when r_reg = 49999999 else
    '0';
  -- next state logic/output logic for second divider
  s_next <= (others=>'0') when (s_reg=59 and s_en='1') else
    s_reg + 1 when s_en='1' else
    s_reg;
  m_en <= '1' when s_reg=59 and s_en='1' else
    '0';
  -- next-state logic for minute divider
  m_next <= (others=>'0') when (m_reg=59 and m_en='1') else
    m_reg + 1 when m_en='1' else
    m_reg;
  -- output logic
  sec <= std_logic_vector(s_reg);
  min <= std_logic_vector(m_reg);
end single_clock_arch;

-- Design Unit que associa a architecture com a entity
CONFIGURATION cfg_ifsc OF timer IS
  FOR multi_clock_arch END FOR;

```

```
--      FOR single_clock_arch END FOR;  
END CONFIGURATION;
```

Esse código descreve um **Cronômetro (Timer) de Segundos e Minutos**. Assim como nos exemplos anteriores, a verdadeira lição desse código não é apenas fazer a contagem, mas sim **como projetar circuitos que dependem da passagem do tempo de forma profissional**.

MULT CLOCK .DO

```
vlib rtl_work  
vmap work rtl_work  
  
vcom -93 -work work {../../list_09_05_06_timer_CORRIGIDO_50mhz.vhd}  
  
vsim work.timer(multi_clock_arch)  
  
add wave -position insertpoint sim:/timer/*  
  
force -freeze sim:/timer/clk 0 0  
force -freeze sim:/timer/reset 1 0  
run  
  
force -freeze sim:/timer/reset 0 0  
run  
  
force -freeze sim:/timer/clk 1 0  
#colocar no ultimo valor de registro de clk e colocar  
force -freeze sim:/timer/r_reg "0101111101011110000011111" 0  
#colocar no ultimo segundo  
force -freeze sim:/timer/s_reg "111011" 0  
run  
  
#tira os valores  
noforce sim:/timer/r_reg  
noforce sim:/timer/s_reg  
  
force -freeze sim:/timer/clk 0 0  
run  
force -freeze sim:/timer/clk 1 0  
run  
force -freeze sim:/timer/clk 0 0  
run  
force -freeze sim:/timer/clk 1 0  
run
```

SINGLE CLOCK .DO

```
vlib rtl_work  
vmap work rtl_work  
  
vcom -93 -work work {../../list_09_05_06_timer_CORRIGIDO_50mhz.vhd}  
  
vsim work.timer(single_clock_arch)  
  
add wave -position insertpoint sim:/timer/*
```

```
radix -unsigned
```

```
force -freeze sim:/timer/clk 0 0  
force -freeze sim:/timer/reset 1 0  
run
```

```
force -freeze sim:/timer/reset 0 0  
run
```

```
force -freeze sim:/timer/r_reg 49999999 0  
force -freeze sim:/timer/s_reg 58 0  
force -freeze sim:/timer/m_reg 58 0  
run
```

```
noforce sim:/timer/r_reg  
noforce sim:/timer/s_reg  
noforce sim:/timer/m_reg
```

```
force -freeze sim:/timer/clk 1 0  
run  
force -freeze sim:/timer/clk 0 0  
run
```

```
force -freeze sim:/timer/r_reg 49999999 0  
run
```

```
noforce sim:/timer/r_reg
```

```
force -freeze sim:/timer/clk 1 0  
run  
force -freeze sim:/timer/clk 0 0  
run
```

VERSÃO DO **YAN**

```
library ieee;  
use ieee.std_logic_1164.all;
```

```
entity bcdtosd is
```

```
    port (  
        sw : in std_logic_vector(3 downto 0);  
        hex0: out bit_vector (6 downto 0)  
    );
```

```
end bcdtosd;
```

architecture code_v1 of bcdtossd is

begin

 process(sw)

 begin

 case sw is

 --abcdefg

 when "0000" => hex0 <= "0000001";

 when "0001" => hex0 <= "1001111";

 when "0010" => hex0 <= "0010010";

 when "0011" => hex0 <= "0000110";

 when "0100" => hex0 <= "1001100";

 when "0101" => hex0 <= "0100100";

 when "0110" => hex0 <= "0100000";

 when "0111" => hex0 <= "0001111";

 when "1000" => hex0 <= "0000000";

 when "1001" => hex0 <= "0001100";

 when others => hex0 <= "0110000";

 end case;

 end process;

end architecture;

configuration ifsc_v1 of bcdtossd is

 for code_v1 end for;

end configuration;

.DO

vlib rtl_work

vmap work rtl_work

vcom -93 -work work {../../bcdtossd.vhd}

vsim work.ifsc_v1

add wave -position insertpoint sim:/bcdtossd/*

force -freeze sim:/bcdtossd/sw 0000 0

run

force -freeze sim:/bcdtossd/sw 0001 0

run

force -freeze sim:/bcdtossd/sw 0010 0

run

force -freeze sim:/bcdtossd/sw 0011 0

run

force -freeze sim:/bcdtossd/sw 0100 0

run

force -freeze sim:/bcdtossd/sw 0101 0

run

force -freeze sim:/bcdtossd/sw 0110 0

run

force -freeze sim:/bcdtossd/sw 0111 0

run

force -freeze sim:/bcdtossd/sw 1000 0

run

force -freeze sim:/bcdtossd/sw 1001 0

run

force -freeze sim:/bcdtossd/sw 1010 0

run

force -freeze sim:/bcdtossd/sw 1011 0

run

force -freeze sim:/bcdtossd/sw 1100 0

run

force -freeze sim:/bcdtossd/sw 1101 0

run

force -freeze sim:/bcdtossd/sw 1110 0

run

force -freeze sim:/bcdtossd/sw 1111 0

run

RESUMO:

Aula 30/04/2026 - Decodificador BCD para Display de 7 Segmentos (bcdtossd)

Objetivo

Implementar um decodificador que recebe um número no formato BCD (Binary-Coded Decimal) e o converte para os sinais correspondentes aos 7 segmentos de um display numérico.

Entradas e Saídas

- Entrada (sw): Chaves de entrada, mapeadas em um vetor de 4 bits.
- Saída (hex0): Saída para o display, usando um vetor de 7 bits (representando os segmentos de 'a' a 'g').

Funcionamento e Lógica

- O circuito utiliza um bloco sequencial process sensível à entrada sw.
- Dentro do processo, a estrutura condicional case mapeia os números BCD (0 a 9). Por exemplo, a entrada "0000" (zero) gera a saída "0000001". Isso indica o acionamento em lógica negativa (o segmento 'g', último bit, está em '1' ou seja, apagado, e os demais formando um "0" estão acesos).
- As chaves de "0000" a "1001" geram os dígitos válidos.
- Há um tratamento when others garantindo estabilidade do circuito que exibirá o padrão "0110000" caso entradas superiores a 9 sejam ativadas.

CÓDIGO 2

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
use ieee.numeric_std.all;
```

```
entity seqDec is
```

```
    port(
```

```
        a: in std_logic_vector (7 downto 0);
```

```
        b: in std_logic_vector (7 downto 0);
```

```

        gt: out std_logic;
        eq: out std_logic;
        lt: out std_logic
    );
end seqDec;

```

architecture exemplo of seqDec is

```

    signal a_signed, b_signed : signed(7 downto 0);

```

```

begin

```

```

    a_signed <= signed(a);
    b_signed <= signed(b);

```

```

    process (a, b)

```

```

        begin

```

```

            if (a > b) then

```

```

                gt <= '1';

```

```

            elsif (a = b) then

```

```

                eq <= '1';

```

```

            else

```

```

                lt <= '1';

```

```

            end if;

```

```

        end process;

```

```

end architecture;

```

architecture correcao1 of seqDec is

```

    signal a_signed, b_signed : signed(7 downto 0);

```

```

begin

```

```

    a_signed <= signed(a);

```

```

    b_signed <= signed(b);

```

```

    process (a_signed, b_signed)

begin
    if (a_signed > b_signed) then
        gt <= '1';
        eq <= '0';
        lt <= '0';
    elsif (a_signed = b_signed) then
        gt <= '0';
        eq <= '1';
        lt <= '0';
    else
        gt <= '0';
        eq <= '0';
        lt <= '1';
    end if;
end process;

end architecture;

architecture correcao2 of seqDec is
    signal a_signed, b_signed : signed(7 downto 0);
begin
    a_signed <= signed(a);
    b_signed <= signed(b);

    process (a_signed, b_signed)
    begin
        gt <= '0';
        eq <= '0';
        lt <= '0';

        if (a_signed > b_signed) then

```

```
        gt <= '1';
    elsif (a_signed = b_signed) then
        eq <= '1';
    else
        lt <= '1';
    end if;
end process;
end architecture;
```

```
configuration seqDec_v1 of seqDec is
    for exemplo end for;
end configuration;
```

```
configuration seqDec_v2 of seqDec is
    for correcao1 end for;
end configuration;
```

```
configuration seqDec_v3 of seqDec is
    for correcao2 end for;
end configuration;
```

. DO

vlib rtl_work

vmap work rtl_work

vcom -93 -work work {../../seqDec.vhd}

vsim work.seqdec_v2

add wave -position insertpoint sim:/seqdec/*

force -freeze sim:/seqdec/a 10000000 0

force -freeze sim:/seqdec/b 10000000 0
run

force -freeze sim:/seqdec/b 00000001 0
run

force -freeze sim:/seqdec/a 11111111 0
force -freeze sim:/seqdec/b 10000000 0
run

force -freeze sim:/seqdec/a 11111011 0
force -freeze sim:/seqdec/b 11111011 0
run

force -freeze sim:/seqdec/a 11111111 0
force -freeze sim:/seqdec/b 00000000 0
run

force -freeze sim:/seqdec/a 00000000 0
force -freeze sim:/seqdec/b 11111111 0
run

.DO 2

vlib rtl_work
vmap work rtl_work

vcom -93 -work work {../../seqDec.vhd}

vsim work.seqdec_v3

add wave -position insertpoint sim:/seqdec/*

force -freeze sim:/seqdec/a 10000000 0
force -freeze sim:/seqdec/b 10000000 0
run
force -freeze sim:/seqdec/b 00000001 0
run

```
force -freeze sim:/seqdec/a 11111111 0
force -freeze sim:/seqdec/b 10000000 0
run
force -freeze sim:/seqdec/a 11111011 0
force -freeze sim:/seqdec/b 11111011 0
run
force -freeze sim:/seqdec/a 11111111 0
force -freeze sim:/seqdec/b 00000000 0
run
force -freeze sim:/seqdec/a 00000000 0
force -freeze sim:/seqdec/b 11111111 0
run
```

```
. DO 3
```

```
vlib rtl_work
vmap work rtl_work
```

```
vcom -93 -work work {../../seqDec.vhd}
```

```
vsim work.seqdec_v1
```

```
add wave -position insertpoint sim:/seqdec/*
```

```
force -freeze sim:/seqdec/a 10000000 0
force -freeze sim:/seqdec/b 10000000 0
run
force -freeze sim:/seqdec/b 00000001 0
run
force -freeze sim:/seqdec/a 11111111 0
force -freeze sim:/seqdec/b 10000000 0
run
force -freeze sim:/seqdec/a 11111011 0
```

```
force -freeze sim:/seqdec/b 11111011 0
run
```

```
force -freeze sim:/seqdec/a 11111111 0
force -freeze sim:/seqdec/b 00000000 0
run
force -freeze sim:/seqdec/a 00000000 0
force -freeze sim:/seqdec/b 11111111 0
run
```

RESUMO: Aula 28/04/2026 - Comparador de Magnitude (Problemas de Latch) (seqDec)

Objetivo

Implementar um comparador de magnitude de 8 bits (Maior que, Igual, Menor que) utilizando processos sequenciais (process). O foco principal desta aula é demonstrar como declarações condicionais incompletas geram latches indesejados e como corrigir esse problema.

Entradas e Saídas

- Entradas: - a, b: Vetores de 8 bits (std_logic_vector).
- Saídas: - gt (Greater Than - Maior que).
 - eq (Equal - Igual).
 - lt (Less Than - Menor que).

Arquiteturas e o Problema do Latch Involuntário

O arquivo seqDec.vhd apresenta três abordagens, mostrando a evolução e correção do código:

1. A Problemática (exemplo): - Utiliza as entradas cruas (a, b) na comparação ao invés dos sinais convertidos para signed.
 - Erro Crítico: No bloco if, a saída gt recebe '1', mas as saídas eq e lt não recebem nada. Como o VHDL precisa manter o valor anterior das saídas não atribuídas, o sintetizador infere um Latch (memória), o que é péssimo para circuitos combinacionais e gera warnings na compilação.
2. A Correção Extensa (correcao1):
 - Compara corretamente os sinais convertidos (a_signed, b_signed).
 - Solução: Em cada ramificação do if, elsif e else, todas as três saídas (gt, eq, lt) recebem explicitamente '0' ou '1'. Isso garante que o circuito seja 100% combinacional, sem memorização de estado.
3. A Correção Elegante (correcao2):
 - Utiliza a técnica de Atribuição Padrão (Default Assignment).
 - Solução: Logo no início do process, antes do if, atribui-se '0' a todas as saídas (gt <= '0'; eq <= '0'; lt <= '0';). Dentro do if, você só precisa se preocupar em mudar para '1' a saída que for verdadeira. O código fica muito mais limpo e imune à criação de latches.

Configurations

Atualmente, a configuração seqDec_v1 está descomentada, instanciando a arquitetura exemplo (com erro) para fins de demonstração. Para testar as soluções na simulação, basta comentar a primeira e descomentar seqDec_v2 ou seqDec_v3 no final do arquivo.

CÓDIGO 3

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity questao1 is
    port
    (
        clk, reset: in std_logic;
        d: in std_logic;
        control: in std_logic_vector (1 downto 0);
        d_to_right: in std_logic;
        d_to_left: in std_logic;
        palavra: in std_logic_vector (3 downto 0);
        q: out std_logic_vector (3 downto 0)
    );
end questao1;

-- 00 para
-- 10 deslocar esquerda
-- 01 deslocar direita
-- 11 pausar

architecture deslocadorUniversal of questao1 is
    signal r_reg : std_logic_vector (3 downto 0);
    signal r_next : std_logic_vector (3 downto 0);
begin

    -- register
```

```

process(clk, reset)
begin
    if (reset = '1') then
        r_reg <= (others => '0');

        elsif (clk'event and clk = '1') then
            r_reg <= r_next;
        end if;
    end process;

process (control, r_reg, palavra, d_to_left, d_to_right)
begin
    case control is
        when "00" => r_next <= r_reg;
        when "01" => r_next <= d_to_right & r_reg (3
downto 1); -- concatena para direita
        when "10" => r_next <= r_reg (2 downto 0) &
d_to_left; -- concatena para a esquerda
        when "11" => r_next <= palavra;
        when others => r_next <= r_reg;
    end case;
end process;

q <= r_reg;
end deslocadorUniversal;

```

.DO

```

vlib rtl_work
vmap work rtl_work
vcom -93 -work work {../../questao1.vhd}
vsim work.questao1(deslocadoruniversal)

```

add wave -position insertpoint sim:/questao1/*

RESET

force -freeze sim:/questao1/reset 1 0
force -freeze sim:/questao1/control 00 0
force -freeze sim:/questao1/palavra 0000 0
force -freeze sim:/questao1/d_to_right 0 0
force -freeze sim:/questao1/d_to_left 0 0
force -freeze sim:/questao1/clk 0 0
run
force -freeze sim:/questao1/clk 1 0
run
force -freeze sim:/questao1/clk 0 0
run

INSERINDO PALAVRA

force -freeze sim:/questao1/reset 0 0
force -freeze sim:/questao1/control 11 0
force -freeze sim:/questao1/palavra 1010 0
force -freeze sim:/questao1/d_to_right 0 0
force -freeze sim:/questao1/d_to_left 0 0
force -freeze sim:/questao1/clk 1 0
run
force -freeze sim:/questao1/clk 0 0
run

DESLOCAR PARA DIREITA BIT 1

force -freeze sim:/questao1/reset 0 0
force -freeze sim:/questao1/control 01 0
force -freeze sim:/questao1/palavra 1010 0

```
force -freeze sim:/questao1/d_to_right 1 0
force -freeze sim:/questao1/d_to_left 0 0
force -freeze sim:/questao1/clk 1 0
run
force -freeze sim:/questao1/clk 0 0
run
```

DESLOCAR PARA DIREITA 0

```
force -freeze sim:/questao1/reset 0 0
force -freeze sim:/questao1/control 01 0
force -freeze sim:/questao1/palavra 1010 0
force -freeze sim:/questao1/d_to_right 0 0
force -freeze sim:/questao1/d_to_left 0 0
force -freeze sim:/questao1/clk 1 0
run
force -freeze sim:/questao1/clk 0 0
run
```

DESLOCAR PARA ESQUERDA BIT 1

```
force -freeze sim:/questao1/reset 0 0
force -freeze sim:/questao1/control 10 0
force -freeze sim:/questao1/palavra 1010 0
force -freeze sim:/questao1/d_to_right 0 0
force -freeze sim:/questao1/d_to_left 1 0
force -freeze sim:/questao1/clk 1 0
run
force -freeze sim:/questao1/clk 0 0
run
```

DESLOCAR PARA DIREITA BIT 0

```
force -freeze sim:/questao1/reset 0 0
```

```
force -freeze sim:/questao1/control 10 0
force -freeze sim:/questao1/palavra 1010 0
force -freeze sim:/questao1/d_to_right 0 0
force -freeze sim:/questao1/d_to_left 0 0
force -freeze sim:/questao1/clk 1 0
run
force -freeze sim:/questao1/clk 0 0
run
```

PAUSAR LEITURA, IGNORA AS ENTRADAS

```
force -freeze sim:/questao1/reset 0 0
force -freeze sim:/questao1/control 00 0
force -freeze sim:/questao1/palavra 1111 0
force -freeze sim:/questao1/d_to_right 1 0
force -freeze sim:/questao1/d_to_left 1 0
force -freeze sim:/questao1/clk 1 0
run
force -freeze sim:/questao1/clk 0 0
run
```

wave zoom full

CÓDIGO 2:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
```

entity questao2 is

```
    port
    (
```

```
    clk, reset: in std_logic;  
    q: out std_logic_vector (2 downto 0)  
    );  
end questao2;
```

architecture contadorSequencial of questao2 is

```
    signal q_contador : std_logic_vector (2 downto 0);  
    signal d_contador : std_logic_vector (2 downto 0);  
begin
```

```
    process (clk,reset)  
    begin  
        if (reset = '1') then  
            q_contador <= "000";  
        elsif (rising_edge(clk)) then  
            q_contador <= d_contador;  
        end if;  
    end process;
```

```
    process (q_contador)  
    begin  
        case q_contador is  
            when "000" => d_contador <= "011";  
            when "011" => d_contador <= "100";  
            when "100" => d_contador <= "101";  
            when "101" => d_contador <= "111";  
            when others => d_contador <= "000";  
        end case;  
    end process;
```

```
    q <= q_contador;
```

end contadorSequencial;

.DO

vlib rtl_work

vmap work rtl_work

vcom -93 -work work {../../questao2.vhd}

vsim work.questao2(contadorsequencial)

add wave -position insertpoint sim:/questao2/*

force -freeze sim:/questao2/reset 1 0

force -freeze sim:/questao2/clk 0 0

run

Passo 2: Libera o Reset (Contador em "000")

force -freeze sim:/questao2/reset 0 0

run

Passo 3: 1º Pulso de Clock -> Deve ir para "011"

force -freeze sim:/questao2/clk 1 0

run

force -freeze sim:/questao2/clk 0 0

run

Passo 4: 2º Pulso de Clock -> Deve ir para "100"

force -freeze sim:/questao2/clk 1 0

run

force -freeze sim:/questao2/clk 0 0

run

Passo 5: 3º Pulso de Clock -> Deve ir para "101"

force -freeze sim:/questao2/clk 1 0

run

force -freeze sim:/questao2/clk 0 0

run

Passo 6: 4º Pulso de Clock -> Deve ir para "111"

force -freeze sim:/questao2/clk 1 0

run

force -freeze sim:/questao2/clk 0 0

run

Passo 7: 5º Pulso de Clock -> Deve resetar a sequência voltando para "000"

force -freeze sim:/questao2/clk 1 0

run

force -freeze sim:/questao2/clk 0 0

run

Passo 8: Teste do Reset Assíncrono no meio do caminho

Dá mais um clock para ele ir para "011" novamente

force -freeze sim:/questao2/clk 1 0

run

force -freeze sim:/questao2/clk 0 0

run

Força o botão de reset (Deve ir para "000" imediatamente, sem precisar de clock)

force -freeze sim:/questao2/reset 1 0

run

wave zoom full

CÓDIGO 3:

library ieee;

use ieee.std_logic_1164.all;

use ieee.numeric_std.all;

entity questao3 is

port

(

clk, reset : in std_logic;

up : in std_logic;

q : out std_logic_vector (7 downto 0)

);

end questao3;

architecture contadorBinario of questao3 is

signal r_reg : unsigned (7 downto 0);

signal r_next : unsigned (7 downto 0);

begin

-- flipflop d

process (clk, reset)

begin

if (reset = '1') then

r_reg <= (others => '0');

elsif (rising_edge(clk)) then

r_reg <= r_next;

end if;

```
end process;
```

```
-- next state logic
```

```
process (up, r_reg)
```

```
begin
```

```
    if (up = '1') then
```

```
        r_next <= r_reg + 1;
```

```
    else
```

```
        r_next <= r_reg - 1;
```

```
    end if;
```

```
end process;
```

```
q <= std_logic_vector(r_reg);
```

```
end contadorBinario;
```

```
.DO
```

```
vlib rtl_work  
vmap work rtl_work
```

```
vcom -93 -work work {../../questao3.vhd}
```

```
vsim work.questao3(contadorbinario)
```

```
add wave -position insertpoint sim:/questao3/*
```

```
# INICIALIZANDO COM RESET
```

```
force -freeze sim:/questao3/reset 1
```

```
force -freeze sim:/questao3/up 1
```

```
force -freeze sim:/questao3/clk 0
```

```
run
```

```
# TIRA O RESET
```

```
force -freeze sim:/questao3/reset 0
```

```
run
```

```
# PRIMEIRO PULSO COMEÇA A CONTAGEM
```

```
force -freeze sim:/questao3/clk 1
```

```
run
```

```
force -freeze sim:/questao3/clk 0
```

```
run
```

```
# SEGUNDO PULSO
```

```
force -freeze sim:/questao3/clk 1
```

```
run
```

```
force -freeze sim:/questao3/clk 0
```

```
run
```

TERCEIRO PULSO

force -freeze sim:/questao3/clk 1

run

force -freeze sim:/questao3/clk 0

run

MUDA CONTAGEM

force -freeze sim:/questao3/up 0

run

QUARTO PULSO

force -freeze sim:/questao3/clk 1

run

force -freeze sim:/questao3/clk 0

run

QUINTO PULSO

force -freeze sim:/questao3/clk 1

run

force -freeze sim:/questao3/clk 0

run

SEXTO PULSO

force -freeze sim:/questao3/clk 1

run

force -freeze sim:/questao3/clk 0

run

SETIMO PULSO

force -freeze sim:/questao3/clk 1

run

force -freeze sim:/questao3/clk 0
run

RESET DO SISTEMA

force -freeze sim:/questao3/reset 1
run

wave zoom full

RESUMO DAS TRES DE CIMA:

Aula 19/05/2026 - Exercícios: Registradores de Deslocamento e Contadores

Objetivo

Este diretório contém a resolução de três questões práticas abordando diferentes tipos de circuitos sequenciais. O foco principal é a separação entre a lógica de estado (flip-flops/registradores) e a lógica combinacional de próximo estado (next-state logic).

Questão 1: Registrador de Deslocamento Universal (Shift Register)

Arquivo: questao1/questao1.vhd

- Objetivo: Implementar um registrador de deslocamento de 4 bits que pode manter o valor, deslocar para a esquerda, deslocar para a direita ou carregar uma nova palavra (carga paralela).
 - Sinais de Controle (control): A estrutura case define a operação com base nos 2 bits do controle:
 - "00": Mantém o estado atual (Pausa).
 - "01": Desloca à direita. Utiliza o operador de concatenação & para juntar o bit de entrada d_to_right com os 3 bits mais significativos do registrador atual (r_reg(3 downto 1)).
 - "10": Desloca à esquerda. Concatena os 3 bits menos significativos (r_reg(2 downto 0)) com o bit de entrada d_to_left.
 - "11": Carga paralela, atualizando o registrador com o valor da entrada palavra.
-

Questão 2: Contador Sequencial de Estados Específicos

Arquivo: questao2/questao2.vhd

- Objetivo: Projetar um contador de 3 bits que não conta de forma linear, mas sim através de uma sequência de estados predefinida.
 - Lógica de Transição: O bloco case dita a seguinte sequência de estados binários (e os seus respectivos valores decimais):
 - 000 (0) -> 011 (3) -> 100 (4) -> 101 (5) -> 111 (7) -> Volta para 000 (0).
 - Tratamento de Falhas: A cláusula when others => d_contador <= "000"; é fundamental. Garante que, se o circuito for inicializado em um estado inválido ou sofrer alguma interferência (estados 1, 2 ou 6), ele se recupere automaticamente para o estado inicial 000 no ciclo de clock seguinte.
-

Questão 3: Contador Binário Crescente/Decrescente (Up/Down)

Arquivo: questao3/questao3.vhd

- Objetivo: Criar um contador de 8 bits capaz de incrementar ou decrementar o valor atual com base em uma entrada de comando.
- Tipos de Dados: Utiliza o tipo unsigned da biblioteca numeric_std para permitir a soma e subtração aritmética direta.
- Lógica de Direção (up):
 - Quando a entrada up = '1', a equação $r_next \leq r_reg + 1$ é executada (conta para cima).
 - Caso contrário (up = '0'), a equação $r_next \leq r_reg - 1$ é executada (conta para baixo).

CÓDIGO:

```
library ieee;
use IEEE.STD_LOGIC_1164.all;

entity mux4x1 is
    Port (
        -- Input
        X : in bit_vector (3 downto 0);
        Sel : in bit_vector (1 downto 0);
        -- Output
        Y : out bit
    );

end mux4x1;

-- boolean logic
architecture v_logica_pura of mux4x1 is
begin
    Y <= (X(0) and (not Sel(1)) and (not Sel(0))) or
        (X(1) and (not Sel(1)) and Sel(0)) or
        (X(2) and Sel(1) and (not Sel(0))) or
        (X(3) and Sel(1) and Sel(0));
end architecture;
```

```
-- when-else
architecture v_when_else of mux4x1 is
begin
    Y<= X(0) when Sel = "00" else
        X(1) when Sel = "01" else
        X(2) when Sel = "10" else
        X(3);
end architecture;
```

```
-- with-select
architecture v_with_select of mux4x1 is
begin
    with Sel select
        Y <= X(0) when "00",
            X(1) when "01",
            X(2) when "10",
            X(3) when others;
end architecture;
```

```
configuration logica_pura of mux4x1 is
    for v_logica_pura end for;
end configuration;
```

```
configuration when_else of mux4x1 is
    for v_when_else end for;
end configuration;
```

```
configuration with_select of mux4x1 is
    for v_with_select end for;
end configuration;
```

```
.DO
vlib rtl_work
vmap work rtl_work

vcom -93 -work work {../../mux4x1.vhd}

vsim work.mux4x1

vsim work.logica_pura

add wave -position insertpoint sim:/mux4x1/*

force -freeze sim:/mux4x1/X 0001 0
run
force -freeze sim:/mux4x1/X 0010 0
force -freeze sim:/mux4x1/Sel 01 0
run
force -freeze sim:/mux4x1/X 0100 0
force -freeze sim:/mux4x1/Sel 10 0
run
force -freeze sim:/mux4x1/X 1000 0
force -freeze sim:/mux4x1/Sel 11 0
run
```

```
. DO 2
```

```
vlib rtl_work
vmap work rtl_work

vcom -93 -work work {../../mux4x1.vhd}
```

```
vsim work.mux4x1
```

```
vsim work.when_else
```

```
add wave -position insertpoint sim:/mux4x1/*
```

```
force -freeze sim:/mux4x1/X 0001 0
```

```
run
```

```
force -freeze sim:/mux4x1/X 0010 0
```

```
force -freeze sim:/mux4x1/Sel 01 0
```

```
run
```

```
force -freeze sim:/mux4x1/X 0100 0
```

```
force -freeze sim:/mux4x1/Sel 10 0
```

```
run
```

```
force -freeze sim:/mux4x1/X 1000 0
```

```
force -freeze sim:/mux4x1/Sel 11 0
```

```
run
```

```
.DO 3
```

```
vlib rtl_work
```

```
vmap work rtl_work
```

```
vcom -93 -work work {../../mux4x1.vhd}
```

```
vsim work.mux4x1
```

```
vsim work.with_select
```

```
add wave -position insertpoint sim:/mux4x1/*
```

```
force -freeze sim:/mux4x1/X 0001 0
run
force -freeze sim:/mux4x1/X 0010 0
force -freeze sim:/mux4x1/Sel 01 0
run
force -freeze sim:/mux4x1/X 0100 0
force -freeze sim:/mux4x1/Sel 10 0
run
force -freeze sim:/mux4x1/X 1000 0
force -freeze sim:/mux4x1/Sel 11 0
run
```

RESUMO DOS 3:

Aula 16/04/2026 - Multiplexador 4x1 (mux4x1)

Objetivo

Demonstrar a implementação de um circuito combinacional Multiplexador (Mux) de 4 entradas e 1 saída através de três diferentes estilos de modelagem (arquiteturas) na linguagem VHDL.

Entradas e Saídas

- Entrada (X): Vetor de 4 bits (3 downto 0) representando os canais de dados.
- Seletor (Sel): Vetor de 2 bits (1 downto 0) para escolher qual canal passar.
- Saída (Y): Bit único que recebe o sinal do canal selecionado.

Arquiteturas Implementadas

Este arquivo explora três sintaxes diferentes para resolver o mesmo problema:

1. Lógica Pura (v_logica_pura): Resolve o multiplexador escrevendo diretamente a equação booleana, utilizando operadores and, or e not.
2. Atribuição Condicional (v_when_else): Utiliza a cláusula concorrente when ... else para direcionar a saída conforme as combinações de "00" a "10" do seletor.
3. Seleção de Sinal (v_with_select): Utiliza a cláusula with Sel select, tornando a leitura da tabela verdade direta e lidando com as alternativas via when e when others.

Configurations

O arquivo finaliza definindo blocos configuration (logica_pura, when_else, with_select) que instruem o compilador/simulador sobre qual arquitetura específica associar à entidade mux4x1 na hora da síntese ou simulação.

CÓDIGO:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity deslocamentoLivre is
    port (
        clk, reset, d: in std_logic;
        q: out std_logic
    );
end deslocamentoLivre;

architecture exemplo of deslocamentoLivre is
    signal r_reg : std_logic_vector (3 downto 0);
    signal r_next : std_logic_vector(3 downto 0);
begin
    -- register
    process(clk, reset)
    begin
        if (reset = '1') then
            r_reg <= (others => '0');
        elsif (clk'event and clk = '1') then
            r_reg <= r_next;
        end if;
    end process;
    -- next-state logic
    r_next <= d & r_reg(3 downto 1);
    -- output logic
    q <= r_reg(0);
end exemplo;
```

. DO

vlib rtl_work

vmap work rtl_work

vcom -93 -work work {../../deslocamentoLivre.vhd}

vsim work.deslocamentolivre(exemplo)

add wave -position insertpoint

sim:/deslocamentolivre/*

force -freeze sim:/deslocamentolivre/reset 1 0
force -freeze sim:/deslocamentolivre/clock 1 0
force -freeze sim:/deslocamentolivre/d 1 0
run

force -freeze sim:/deslocamentolivre/reset 0 0
force -freeze sim:/deslocamentolivre/clock 0 0
run

force -freeze sim:/deslocamentolivre/clock 1 0
run

force -freeze sim:/deslocamentolivre/clock 0 0
run

force -freeze sim:/deslocamentolivre/clock 1 0
run

force -freeze sim:/deslocamentolivre/d 0 0
force -freeze sim:/deslocamentolivre/clock 0 0
run

force -freeze sim:/deslocamentolivre/clock 1 0
run

force -freeze sim:/deslocamentolivre/clock 0 0
run

force -freeze sim:/deslocamentolivre/clock 1 0

run

wave zoom full

CÓDIGO 2:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity contadorDecimal is
    port (
        clk, reset: in std_logic;
        q: out std_logic_vector(3 downto 0)
    );
end contadorDecimal;

architecture exemplo of contadorDecimal is
    constant DEZ : integer := 10;
    signal r_reg : unsigned (3 downto 0);
    signal r_next : unsigned (3 downto 0);
begin
    -- register
    process(clk, reset)
    begin
        if (reset = '1') then
            r_reg <= (others => '0');
        elsif (clk'event and clk = '1') then
            r_reg <= r_next;
        end if;
    end process;
end;
```

```
        end if;
    end process;
    -- next-state logic
    r_next <= (others => '0') when r_reg = (DEZ - 1)
else
        r_reg + 1;
    -- output logic
    q <= std_logic_vector(r_reg);
end exemplo;
```

. DO

vlib rtl_work

vmap work rtl_work

vcom -93 -work work {../../contadorDecimal.vhd}

vsim work.contadordecimal(exemplo)

add wave -position insertpoint

sim:/contadordecimal/*

força reset e clock em 0 para começar

force -freeze sim:/contadordecimal/reset 1 0

force -freeze sim:/contadordecimal/clock 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

force -freeze sim:/contadordecimal/reset 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

**force -freeze sim:/contadordecimal/clk 1 0
run**

**force -freeze sim:/contadordecimal/clk 0 0
run**

**force -freeze sim:/contadordecimal/clk 1 0
run**

**force -freeze sim:/contadordecimal/clk 0 0
run**

**force -freeze sim:/contadordecimal/clk 1 0
run**

**force -freeze sim:/contadordecimal/clk 0 0
run**

**force -freeze sim:/contadordecimal/clk 1 0
run**

**force -freeze sim:/contadordecimal/clk 0 0
run**

**force -freeze sim:/contadordecimal/clk 1 0
run**

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

force -freeze sim:/contadordecimal/clk 0 0

run

force -freeze sim:/contadordecimal/clk 1 0

run

```
force -freeze sim:/contadordecimal/clk 0 0  
run
```

```
force -freeze sim:/contadordecimal/clk 1 0  
run
```

```
force -freeze sim:/contadordecimal/clk 0 0  
run
```

```
wave zoom full
```

CÓDIGO 3:

```
library ieee;  
use ieee.std_logic_1164.all;  
use ieee.numeric_std.all;  
  
entity contadorBinario is  
    port (  
        clk, reset: in std_logic;  
        max_pulse: out std_logic;  
        q: out std_logic_vector(3 downto 0)  
    );  
end contadorBinario;  
  
architecture exemplo of contadorBinario is  
    signal r_reg : unsigned (3 downto 0);  
    signal r_next : unsigned (3 downto 0);  
begin
```

```

-- register
process(clk, reset)
begin
    if (reset = '1') then
        r_reg <= (others => '0');
    elsif (clk'event and clk = '1') then
        r_reg <= r_next;
    end if;
end process;
-- next-state logic
r_next <= r_reg + 1;
-- output logic
q <= std_logic_vector(r_reg);
max_pulse <= '1' when r_reg = "1111"
                else '0';
end exemplo;

```

. DO

```

vlib rtl_work
vmap work rtl_work
vcom -93 -work work {../contadorBinario.vhd}
vsim work.contadorbinario(exemplo)

```

```

add wave -position insertpoint
sim:/contadorbinario/*

```

```

force -freeze sim:/contadorbinario/reset 1 0
force -freeze sim:/contadorbinario/clk 1 0

```

run

force -freeze sim:/contadorbinario/clock 0 0

force -freeze sim:/contadorbinario/reset 0 0

run

force -freeze sim:/contadorbinario/clock 1 0

run

force -freeze sim:/contadorbinario/clock 0 0

run

force -freeze sim:/contadorbinario/clock 1 0

run

force -freeze sim:/contadorbinario/clock 0 0

run

force -freeze sim:/contadorbinario/clock 1 0

run

force -freeze sim:/contadorbinario/clock 0 0

run

force -freeze sim:/contadorbinario/clock 1 0

run

force -freeze sim:/contadorbinario/clock 0 0

run

force -freeze sim:/contadorbinario/clock 1 0
run

force -freeze sim:/contadorbinario/clock 0 0
run

force -freeze sim:/contadorbinario/clock 1 0
run

force -freeze sim:/contadorbinario/clock 0 0
run

force -freeze sim:/contadorbinario/clock 1 0
run

force -freeze sim:/contadorbinario/clock 0 0
run

force -freeze sim:/contadorbinario/clock 1 0
run

force -freeze sim:/contadorbinario/clock 0 0
run

force -freeze sim:/contadorbinario/clock 1 0
run

force -freeze sim:/contadorbinario/clock 0 0
run

force -freeze sim:/contadorbinario/clock 1 0
run

force -freeze sim:/contadorbinario/clock 0 0
run

force -freeze sim:/contadorbinario/clock 1 0
run

force -freeze sim:/contadorbinario/clock 0 0
run

force -freeze sim:/contadorbinario/clock 1 0
run

force -freeze sim:/contadorbinario/clock 0 0
run

force -freeze sim:/contadorbinario/clock 1 0
run

force -freeze sim:/contadorbinario/clock 0 0
run

force -freeze sim:/contadorbinario/clock 1 0
run

force -freeze sim:/contadorbinario/clock 0 0

run

force -freeze sim:/contadorbinario/clock 1 0

run

force -freeze sim:/contadorbinario/clock 0 0

run

force -freeze sim:/contadorbinario/clock 1 0

run

wave zoom full

RESUMO:

Aula 12/05/2026 - Contador Binário (contadorBinario)

Objetivo

Projetar um contador binário sequencial de 4 bits utilizando o tipo de dado unsigned da biblioteca numeric_std.

Entradas e Saídas

- Entradas: clock e reset.
- Saídas: - q: Vetor de 4 bits contendo o valor atual da contagem.
 - max_pulse: Um pulso ("flag") ativado apenas quando o contador atinge o valor máximo.

Arquitetura e Modelagem

A arquitetura exemplo adota a separação de lógica de estado (registradores) e lógica de transição, uma excelente prática em projetos digitais:

- Registradores (r_reg, r_next): Sinais do tipo unsigned (4 bits) armazenam o valor atual e o próximo.

- Lógica Sequencial (Estado): Um processo sensível ao clk e reset. O reset assíncrono zera a contagem com o comando (others => '0'). As mudanças regulares ocorrem sempre em transições de subida (clk'event and clk = '1'), atualizando o registrador (r_reg <= r_next).
- Lógica de Próximo Estado: A equação concorrente r_next <= r_reg + 1 garante que o contador aumente a cada ciclo.
- Sinalizador de Máximo: A saída max_pulse é combinacional e sobe para '1' somente na condição em que o estado é "1111" (15 decimal).

CÓDIGO:

```
library ieee;
use ieee.std_logic_1164.all;

entity flipFlopEnable is

    port (
        rst : in std_logic;
        clk : in std_logic;
        enable : in std_logic;
        d : in std_logic;
        q : out std_logic
    );

end flipFlopEnable;

architecture ffde of flipFlopEnable is
begin
    process (clk, rst)
    begin
        if rst = '1' then
            q <= '0';
```

```
        elsif rising_edge(clk) then -- rising edge
sensitive
            if enable = '1' then
                q <= d;
            end if;
        end if;
    end process;
end architecture;
```

```
configuration ifsc_v1 of flipFlopEnable is
    for ffde end for;
end configuration;
```

```
.DO
```

```
vlib rtl_work
```

```
vmap work rtl_work
```

```
vcom -93 -work work {../..}/flipFlopEnable.vhd}
```

```
vsim work.ifsc_v1
```

```
add wave -position insertpoint sim:/flipflopenable/*
```

```
force -freeze sim:/flipflopenable/rst 1 0
```

force -freeze sim:/flipflopenable/clk 0 0
force -freeze sim:/flipflopenable/enable 0 0
force -freeze sim:/flipflopenable/d 0 0
run 20ns

force -freeze sim:/flipflopenable/rst 0 0
force -freeze sim:/flipflopenable/enable 1 0
force -freeze sim:/flipflopenable/d 1 0
run 10ns
force -freeze sim:/flipflopenable/clk 1 0
run 20ns

force -freeze sim:/flipflopenable/clk 0 0
force -freeze sim:/flipflopenable/enable 0 0
force -freeze sim:/flipflopenable/d 0 0
run 20ns
force -freeze sim:/flipflopenable/clk 1 0
run 20ns

force -freeze sim:/flipflopenable/clk 0 0
force -freeze sim:/flipflopenable/enable 1 0
run 20ns
force -freeze sim:/flipflopenable/clk 1 0
run 20ns

wave zoom full

RESUMO:

Objetivo

Construir um Flip-Flop tipo D equipado com um controle de Enable (Habilitação) síncrono e um Reset assíncrono.

Sinais da Entidade

- Entradas:
 - clk (Clock), rst (Reset), enable (Habilitação), d (Dado).
- Saídas:
 - q (Saída do registrador).

Funcionamento

O Flip-Flop processa os sinais clk e rst em sua lista de sensibilidade:

1. Prioridade do Reset: Se rst = '1', a saída q vai imediatamente para '0', ignorando o relógio (comportamento assíncrono).
2. Atualização Síncrona: Apenas na borda de subida do relógio (elsif rising_edge(clk)), o circuito verifica a condição de habilitação.
3. Enable: O dado d só é transferido para a saída q se o pino enable estiver em nível lógico alto ('1').

CÓDIGO:

```
library ieee;
use ieee.std_logic_1164.all;

entity seqAForLoop is
    port (
        a : in std_logic_vector(7 downto 0);
        p : inout std_logic_vector(7 downto 0);

        y : out std_logic
    );
end seqAForLoop;
```

```
architecture code_v1 of seqAForLoop is
  begin
    process(a, p)
      begin
        p(0) <= a(0);

        for i in 1 to 7 loop
          p(i) <= a(i) xor p(i - 1);
        end loop;

        y <= p(7);
      end process;
    end architecture;
```

```
configuration ifsc_v1 of seqAForLoop is
  for code_v1 end for;
end configuration;
```

.DO

```
vlib rtl_work
```

```
vmap work rtl_work
```

```
vcom -93 -work work {seqAForLoop.vhd}
```

```
vsim work.seqaforloop(code_v1)
```

```
add wave -position insertpoint sim:/seqaforloop/*
```

```
force -freeze sim:/seqaforloop/a 00000000 0
```

```
run
```

```
force -freeze sim:/seqaforloop/a 00000001 0
```

```
run
```

```
force -freeze sim:/seqaforloop/a 10000001 0
```

```
run
```

```
force -freeze sim:/seqaforloop/a 11111111 0
```

```
run
```

RESUMO: Aula 05/05/2026 - Verificador de Paridade com For Loop (seqAForLoop)

Objetivo

Demonstrar a implementação de um circuito Verificador de Paridade utilizando estruturas de repetição (for loop) dentro de um bloco sequencial (process) na linguagem VHDL.

Entradas e Saídas

- Entrada: Um vetor de dados (normalmente um std_logic_vector de tamanho parametrizável ou fixo, como 8 bits) que será analisado.
- Saídas: Um sinal de bit (ou dois, para indicar explicitamente paridade par ou ímpar) que reflete o cálculo da paridade dos bits de entrada.

Arquitetura e Modelagem (seqAForLoop.vhd)

A modelagem comportamental presente neste arquivo utiliza a repetição para criar um circuito que, na prática, é combinacional:

- Laço de Repetição (for ... loop): Em vez de escrever a operação manual para cada bit (ex: $a(0) \text{ xor } a(1) \text{ xor } a(2) \dots$), o código itera sobre todos os índices do vetor de entrada.
- Acumulação com XOR: Dentro do laço, uma variável ou sinal acumulador recebe a operação lógica xor entre o seu valor atual e o próximo bit do vetor. Se a quantidade de bits '1' no vetor de entrada for ímpar, o resultado final do XOR será '1'.
- Síntese de Laços: É fundamental compreender que, como não há sinal de clock (relógio) controlando este process, o sintetizador "desenrola" (unroll) esse laço for durante a compilação, gerando uma grande porta lógica combinacional em cascata (ou árvore de XORs).

Simulação e Estrutura do Projeto

- tb_paridade.do: Arquivo de macro para o ModelSim. Ele contém os comandos de estímulo (forces) aplicados à entrada do circuito para varrer diferentes combinações de bits e verificar se a saída de paridade se comporta conforme o esperado na forma de onda.
- verificadorParidade.qar: Arquivo de projeto arquivado (Quartus Archive). Contém toda a base do projeto já configurada, bastando restaurá-lo no Quartus para ter o ambiente pronto.